



本小节内容

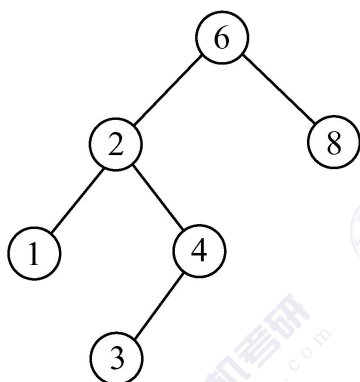
二叉排序树原理解析

二叉排序树代码实战

二叉排序树原理解析

二叉排序树（也称二叉查找树）或者是一棵空树，或者是具有下列特性的二叉树：

- 1) 若左子树非空，则左子树上所有结点的值均小于根结点的值。
- 2) 若右子树非空，则右子树上所有结点的值均大于根结点的值。
- 3) 左、右子树也分别是一棵二叉排序树。



通过网址 <https://www.cs.usfca.edu/~galles/visualization/BST.html> 来演示

二叉排序树动画效果，如果左键点击 pdf 网址不能打开网页的同学，可以直接到

QQ 群公告获取网址

二叉排序树代码实战

代码实战步骤：首先我们新建了一颗二叉排序树，使用的是非递归的方法新建，当然递归的代码也给出了，不过注释了。然后针对建好的二叉排序树进行了



中序遍历输出，接着对二叉排序树进行查找，我们可以看到二叉排序树的最大查找次数是树的高度。理解代码时可以结合上面的动画网站。

```
#include <stdio.h>
#include <stdlib.h>

typedef int KeyType;
typedef struct BSTNode{
    KeyType key;
    struct BSTNode *lchild,*rchild;
}BSTNode,*BiTree;

//54,20,66,40,28,79,58
// 王道书上的递归写法，代码简单，但是理解有难度
//int BST_Insert(BiTree &T,KeyType k)
//{
//    if(NULL==T)
//    {    //为新节点申请空间，第一个结点作为树根，后面递归再进入的不是树根，是为叶子结点
//        T=(BiTree)malloc(sizeof(BSTNode));
//        T->key=k;
//        T->lchild=T->rchild=NULL;
//        return 1;//代表插入成功
//    }
//    else if(k==T->key)
//        return 0;//发现相同元素，就不插入
//    else if(k<T->key)//如果要插入的结点，小于当前结点
//        //函数调用结束后，左孩子和原来的父亲会关联起来，巧妙利用了引用机制
//        return BST_Insert(T->lchild,k);
//    else
//        return BST_Insert(T->rchild,k);
//}
//54,20,66,40,28,79,58
int BST_Insert(BiTree& T, KeyType k)
{
    if (NULL == T)
    {    //为新节点申请空间，第一个结点作为树根，T 是树根
        T = (BiTree)malloc(sizeof(BSTNode));
        T->key = k;
        T->lchild = T->rchild = NULL;
        return 1;//代表插入成功
    }
    BiTree p = T,parent;//p 用来遍历,parent 用来存 p 的父亲
    while (p)
    {
```



```
parent = p;
if (k == p->key)
{
    return 0;
}
else if (k < p->key)
{
    p = p->lchild;
}
else {
    p = p->rchild;
}
}
//开始申请空间并插入
BiTree pnew = (BiTree)malloc(sizeof(BSTNode));
pnew->key = k;
pnew->lchild = pnew->rchild = NULL;
if (k > parent->key)
{
    parent->rchild = pnew; //新结点在父亲的右边
}
else {
    parent->lchild = pnew; //新结点在父亲的左边
}
}
```

//创建二叉排序树

```
void Creat_BST(BiTree &T, KeyType str[], int n)
{
```

```
    T=NULL; //T 是树根
    int i=0;
    while(i<n)
    {
        BST_Insert(T, str[i]); //把某一个结点放入二叉查找树
        i++;
    }
}
```

//也可以用递归实现，只不过循环查找已经非常简单清晰，用递归必要性不强

```
BSTNode *BST_Search(BiTree T, KeyType key, BiTree &p)
```

```
{
    p=NULL; //存储要找的结点的父亲
    while(T!=NULL && key!=T->key)
    {
        p=T;
    }
}
```



```
        if(key<T->key) T=T->lchild;//比当前节点小，就左边找
        else T=T->rchild;//比当前节点大，右边去
    }
    return T;
}

void InOrder(BiTree T)
{
    if(T!=NULL)
    {
        InOrder(T->lchild);
        printf("%3d",T->key);
        InOrder(T->rchild);
    }
}

//《王道 C 督学营》课程
//二叉排序树的创建，中序遍历，查找，删除
int main()
{
    BiTree T=NULL;//树根
    BiTree parent;//存储父亲结点的地址值
    BiTree search;
    KeyType str[7]={54,20,66,40,28,79,58};//将要进入二叉排序树的元素值
    Creat_BST(T,str,7);
    InOrder(T);
    printf("\n");
    search=BST_Search(T,40,parent);
    if(search)
    {
        printf("找到对应结点， 值=%d\n",search->key);
    }else{
        printf("未找到对应结点\n");//没找到 search 返回的是 NULL
    }
    return 0;
}
```